

Phát triển ứng dụng web

ReactJS
(part 3)

Nội dung

- ☐ ReactJS – UI Library
- ☐ ReactJS – Forms
- ☐ ReactJS – Context

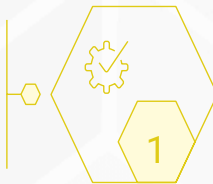
Nội dung

- ❑ **ReactJS – UI Library**
- ❑ ReactJS – Forms
- ❑ ReactJS – Context

Khó khăn khi làm UI cho ứng dụng

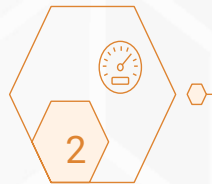
Tính nhất quán

Đảm bảo các thành phần UI trông giống nhau trên các trang khác nhau và quản lý các theme.



Tốc độ

Tránh tốn thời gian code các thành phần UI phức tạp như modal và dropdown.



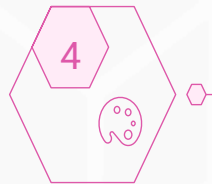
Khả năng truy cập

Đảm bảo người dùng có thể tương tác với UI bằng bàn phím và trình đọc màn hình.



Khả năng tùy biến

Cho phép tùy chỉnh các thành phần UI để đáp ứng các yêu cầu cụ thể của khách hàng.



UI Library

- ☐ UI Library là một **bộ sưu tập các component (thành phần) đã được code và tạo kiểu (style) sẵn**, sẵn sàng để sử dụng.
- ☐ Mục tiêu chính của UI Library là:
 - ☐ Tăng Tốc độ phát triển ứng dụng.
 - ☐ Đảm bảo Tính nhất quán.
 - ☐ Giải quyết A11y (Accessibility).
- ☐ Có 4 trường phái chính.

Component Libraries

- ☐ Đây là loại phổ biến nhất. Chúng cung cấp các component đã hoàn thiện 100% cả về **Giao diện (Style)** lẫn **Logic (Functionality)** và **A11y**.
- ☐ **Ví dụ tiêu biểu: Material-UI (MUI), Ant Design (AntD), Chakra UI,...**
- ☐ Ưu điểm:
 - ☐ Cực kỳ nhanh
 - ☐ Đồng bộ tuyệt đối
 - ☐ Hệ sinh thái lớn
- ☐ Nhược điểm:
 - ☐ Rất khó tùy biến.
 - ☐ Bundle size lớn
 - ☐ Nhận diện thương hiệu

Utility-First CSS

- ❑ Đây *không hẳn* là một thư viện *component*. Nó là một thư viện các "lớp" (class) CSS nguyên tử (atomic).
- ❑ **Ví dụ tiêu biểu: Tailwind CSS.**
- ❑ Ưu điểm:
 - ❑ Tùy biến 100%
 - ❑ Performance tốt
 - ❑ Tốc độ code
- ❑ Nhược điểm:
 - ❑ Chỉ lo Giao diện, KHÔNG lo Logic/A11y
 - ❑ "HTML bẩn"

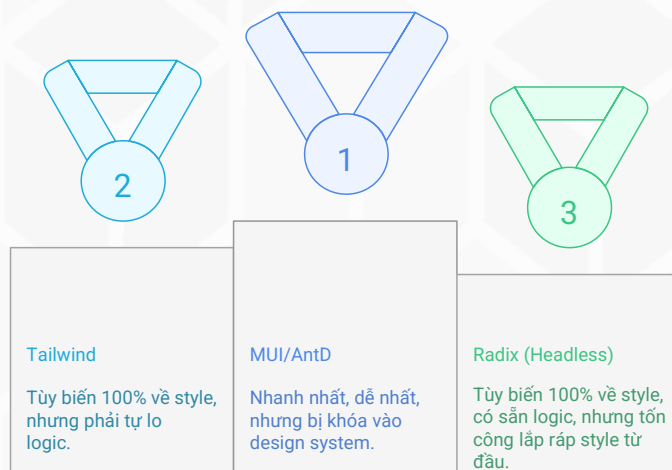
Headless UI

- ❑ Đây là một khái niệm cực kỳ mạnh mẽ. Nó cung cấp các component "vô hình". Chúng có đầy đủ *logic*, *trạng thái* và *A11y*, nhưng hoàn toàn **KHÔNG** có một dòng CSS nào.
- ❑ **Ví dụ tiêu biểu: Radix Primitives, React Aria, Headless UI (của team Tailwind).**
- ❑ Ưu điểm:
 - ❑ Kết hợp tốt nhất của 2 thế giới
 - ❑ Tùy biến Giao diện 100%
- ❑ Nhược điểm:
 - ❑ Tốn công nhất

CSS-in-JS

- ❑ Đây là một phương pháp cho phép viết CSS thực thụ ngay trong file JavaScript. Style được "scope" (đóng gói) vào component và có thể dễ dàng thay đổi dựa trên props.
- ❑ **Ví dụ tiêu biểu: Styled-Components, Emotion.**
- ❑ Ưu điểm: Rất mạnh mẽ cho việc tạo style động (ví dụ: `<Button $primary>`). Không bao giờ lo xung đột CSS.
- ❑ Nhược điểm: Có thể có "runtime overhead" (ảnh hưởng performance một chút) vì style được tính toán lúc chạy.

Mỗi trường phái mỗi hạn chế



➡ Kết hợp ưu điểm cả 3?

Shadcn UI - "Con lai" hoàn hảo

- ❑ **Shadcn UI KHÔNG PHẢI** là một thư viện component.
 - ❑ Nó là một **bộ sưu tập (collection)** các component đã được *build* sẵn, sử dụng **Tailwind CSS** (cho giao diện) và **Radix Primitives** (cho logic/a11y).
- ❑ Shadcn mang lại:
 - ❑ Toàn quyền tùy biến
 - ❑ Gọn nhẹ (Lightweight)
 - ❑ A11y có sẵn
 - ❑ Dễ dàng quản lý

Chuẩn bị trước khi cài đặt Shadcn

- ❑ Thêm **alias** vào **vite.config.js**

```
resolve: {  
  alias: {  
    "@": path.resolve(__dirname, "./src"),  
  },  
},
```

- ❑ Tạo file **jsconfig.json** với nội dung

```
{  
  "compilerOptions": {  
    "baseUrl": ".",  
    "paths": {  
      "@/*": ["./*"]  
    }  
  }  
}
```

- ❑ Đã cài đặt đầy đủ TailwindCSS 4 cho VITE trước khi thực hiện init Shadcn

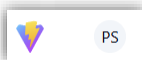
Khởi tạo Shadcn

- ❑ Khởi tạo Shadcn bằng lệnh sau:
`npx shadcn@latest init`
- ❑ Shadcn sẽ tự detect cấu hình của project để thiết lập môi trường phù hợp.
 - ❑ Phù hợp build tool (VITE)
 - ❑ Nơi đặt các ui, lib, biến theme (tailwindcss),...
 - ❑ Ngôn ngữ: typescript hay javascript.

Sử dụng

- ❑ Cần component nào thì kéo component đó về project bằng lệnh:
`npx shadcn@latest add <component>`
- ❑ Ví dụ: **`npx shadcn@latest add avatar`**

```
<Avatar>
  <AvatarImage src="/vite.svg" alt="@shadcn" />
</Avatar>
<Avatar className="rounded-lg">
  <AvatarImage src="person.png" alt="@person" />
  <AvatarFallback>PS</AvatarFallback>
</Avatar>
```



Kết hợp sử dụng icon với lucide-react

- ❑ Chỉ cần import, VITE sẽ tự thêm vào project mà không cần install thủ công "lucide-react"

```
import { CheckLine, DiamondPlus } from "lucide-react";
```

```
<Button  
  variant="outline"  
  size="lg"  
  onClick={() => setCount((count) => count + 1)}  
>  
  <Icons.diamondPlus className="mr-2 h-4 w-4" />  
  {count}  
</Button>
```

13

Áp dụng

- ❑ Xây dựng chức năng login/logout/register và profile cho ứng dụng.

Login

Sign Up

Username

un01

Password

.....

Login

NV Nguyễn Văn A

Nguyễn Văn A
nguyenvana@example.com

Profile

Settings

Log out

Profile

Username: un01

Name: Nguyễn Văn A

Email: nguyenvana@example.com

Logout

Nội dung

- ☐ *ReactJS – UI Library*
- ☐ **ReactJS – Forms**
- ☐ ReactJS – Context

Chức năng Register đơn giản

Sign Up
Create an account to access all features.

Username

Email

Password

Confirm Password

Vấn đề: Quản lý state và Boilerplate Code

```
const [username, setUsername] = useState("");
const [email, setEmail] = useState("");
const [password, setPassword] = useState("");
const [confirmPassword, setConfirmPassword] = useState("");
const [errors, setErrors] = useState({});
```

```
<input
  type="text"
  value={username}
  onChange={(e) => setUsername(e.target.value)}
/>
<input
  type="email"
  value={email}
  onChange={(e) => setEmail(e.target.value)}
/>
```

Vấn đề: hiệu năng và validation

- ☐ Quá nhiều re-render?
- ☐ Thực hiện validation khi nào?
 - ☐ onChange
 - ☐ onBlur
 - ☐ onSubmit
- ☐ Cách thức phản hồi lỗi?
- ☐ Quản lý logic khó khăn.

Vấn đề: quản lý trạng thái Form

- ❑ Làm sao để vô hiệu hóa (disable) nút Submit khi form đang được submit (để tránh double-click)?
- ❑ Làm sao để vô hiệu hóa nút Submit khi form *chưa hợp lệ* (invalid)?

Controlled vs Uncontrolled Component

- ❑ Controlled:
 - ❑ **Value** trong **state React**
 - ❑ `value={state}` + `onChange={setState}`
- ❑ Uncontrolled:
 - ❑ DOM quản lý value (`defaultValue`, `ref`)
- ❑ Thư viện form:
 - ❑ **Formik**: thiên về controlled
 - ❑ **react-hook-form**: tận dụng uncontrolled + ref để tối ưu

Tổng quan react-hook-form

- ❑ Triết lý: “Minimally re-render, maximize performance”

- ❑ Key APIs:

- ❑ `useForm()` → trả về `register`, `handleSubmit`, `formState`, `control`, ...

- ❑ `register("fieldName")` để “đăng ký” input

- ❑ `handleSubmit(onValid, onInvalid)` để xử lý submit

- ❑ Ưu điểm:

- ❑ Ít re-render, nhẹ

- ❑ Dễ tích hợp với UI library

- ❑ Có resolver cho Yup/Zod

Cài đặt

- ❑ Cài đặt các thư viện cần thiết:

- `npm i react-hook-form`

- `npm i zod @hookform/resolvers`

Validation nâng cao với Zod

❑ Tạo schema validation

❑ Tách logic validation ra khỏi UI

```
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";

const schema = z
  .object({
    username: z.string().max(30).nonempty("Tên đăng nhập là bắt buộc"),
    email: z.email("Email không hợp lệ"),
    password: z.string().min(8, "Mật khẩu phải có ít nhất 8 ký tự"),
    confirmPassword: z
      .string()
      .min(8, "Xác nhận mật khẩu phải có ít nhất 8 ký tự"),
  })
  .refine((data) => data.password === data.confirmPassword, {
    message: "Mật khẩu không khớp",
    path: ["confirmPassword"],
  });
```

Tích hợp react-hook-form với UI Library

```
const {
  register,
  handleSubmit,
  formState: { errors, isSubmitting },
} = useForm({
  resolver: zodResolver(schema),
  mode: "onBlur",
});
```

```
<div className="grid gap-2">
  <Label htmlFor="username">Username</Label>
  <Input
    id="username"
    type="text"
    placeholder="Enter your username"
    {...register("username")}
  />
  {errors.username && (
    <p style={{ color: "red" }}>{errors.username.message}</p>
  )}
</div>
```

Nội dung

- ☐ *ReactJS – UI Library*
- ☐ *ReactJS – Forms*
- ☐ **ReactJS – Context**

Prop Drilling & Global State đơn giản

- ☐ Prop drilling là gì?
 - ☐ Truyền props qua nhiều tầng component chỉ để một component con dùng
- ☐ Nhược điểm:
 - ☐ Khó bảo trì
 - ☐ Khó refactor (di chuyển component)
 - ☐ Gây re-render không cần thiết
- ☐ Ví dụ: Avatar ở Navbar

 MyShop Product showcase

NV Nguyễn Văn A

React Context

- ❑ **React Context** cung cấp một cách để chia sẻ dữ liệu ("state") giữa các **component** mà không cần phải truyền **props** một cách tường minh qua từng cấp của cây **component**.
- ❑ Kịch bản chính:
 - ❑ Tạo kênh phân phối toàn cục.
 - ❑ Bất kỳ component con nào ở trong kênh phân phối dù sâu bao nhiêu đều có thể truy suất trực tiếp dữ liệu từ kênh đó.

3 bước sử dụng Context API

- ❑ **TẠO (Create):**
 - ❑ Sử dụng **React.createContext()** để tạo một đối tượng Context.
- ❑ **CUNG CẤP (Provide):**
 - ❑ Sử dụng component **Context.Provider** để bọc cây component cần truy cập dữ liệu.
- ❑ **SỬ DỤNG (Consume):**
 - ❑ Sử dụng Hook **useContext()** trong một functional component con.

Bước 1: `React.createContext(defaultValue)`

- ❑ Tạo React component để cung cấp context và provider

```
const AuthContext = createContext(null);
export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const value = {
    user,
    isAuthenticated: !!user,
    login,
    logout,
  };
  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}
```

```
export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error("useAuth must be used within an AuthProvider");
  }
  return context;
}
```

Bước 2: `Context.Provider`

- ❑ Sử dụng component Provider (là một thuộc tính của object Context).
- ❑ Bọc component này quanh cây component cần truy cập dữ liệu.
- ❑ Prop quan trọng nhất là value - đây chính là dữ liệu muốn chia sẻ.

```
createRoot(document.getElementById("root")).render(
  <StrictMode>
    <AuthProvider>
      <App />
    </AuthProvider>
  </StrictMode>
);
```


Bước 3: `useContext(Context)`

- ❑ Đây là Hook để "tiêu thụ" (đọc) giá trị từ Context.
- ❑ Nó nhận vào chính đối tượng Context (không phải Provider).
- ❑ Nó trả về giá trị value từ Provider gần nhất.
- ❑ Quan trọng: **`useContext`** đăng ký (subscribes) component này với Context. Khi value của Provider thay đổi, component này sẽ re-render.

Ví dụ

```
import { useAuth } from "@/context/auth";
```

```
const { user, logout, isAuthenticated } = useAuth();
```

```
<Avatar className="h-7 w-7">  
  <AvatarImage src={user.avatarUrl} alt={user.name} />  
  <AvatarFallback>  
    {user.name  
      .split(" ")  
      .map((p) => p[0])  
      .join("")  
      .slice(0, 2)}  
  </AvatarFallback>  
</Avatar>
```

Nội dung

- ❑ *ReactJS – UI Library*
- ❑ *ReactJS – Forms*
- ❑ *ReactJS – Context*
- ❑ **Bài tập**

Bài tập

- ❑ Với dữ liệu được cung cấp từ file **data.json**, hãy xây dựng ứng dụng web có các chức năng: register, login, logout, profile và product showcase có phân trang.

